北京理工大學
BEIJING INSTITUTE OF TECHNOLOGY

类的组合和继承

本章知识点

- 类与类的关系
- 类的组合
- 类的继承
  - 继承的概念和作用
  - 子类的声明
  - 类成员修饰符与继承的关系
  - 继承机制下对象的创建
  - 子类对父类方法的重写(super调用)

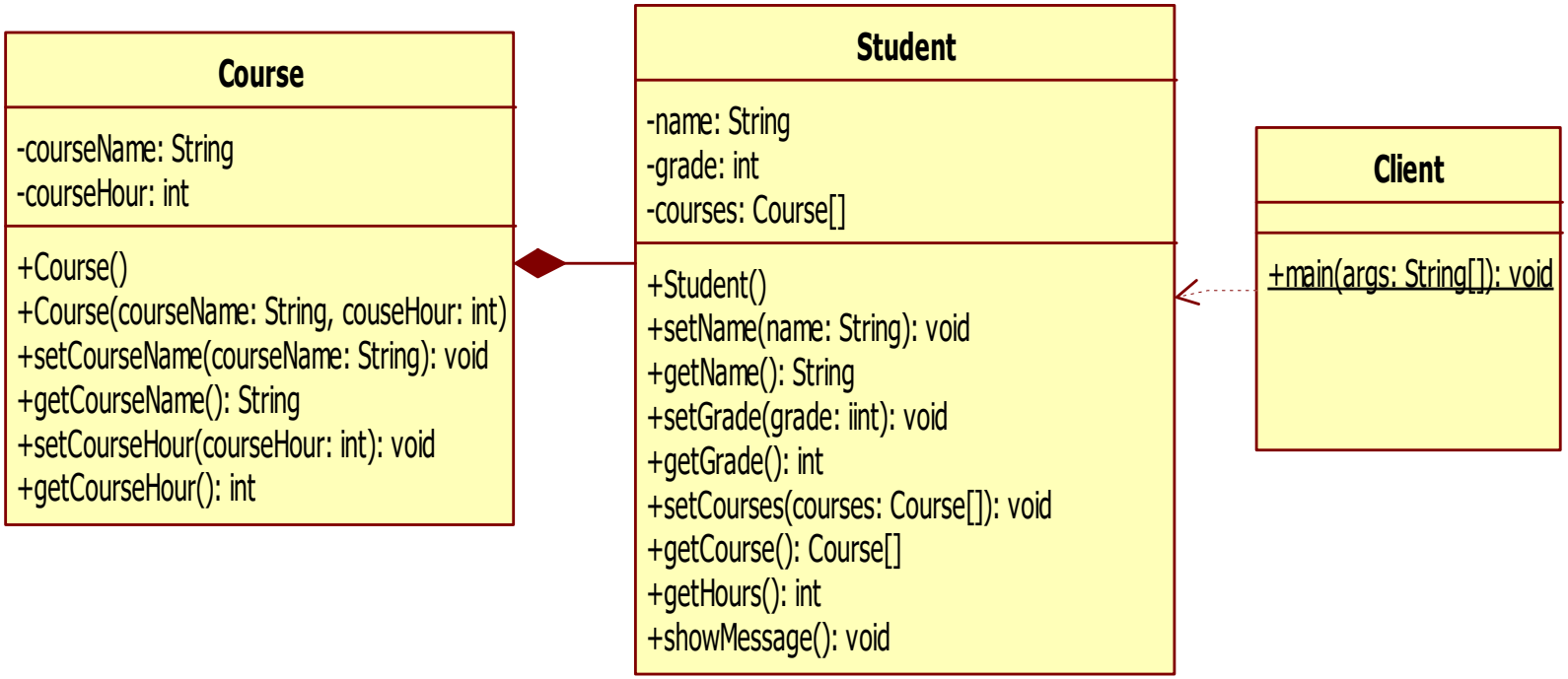
- 类与类的关系
- 类的组合
- 类的继承
  - 继承的概念和作用
  - 子类的声明
  - 类成员修饰符与继承的关系
  - 继承机制下对象的创建
  - 子类对父类方法的重写(super调用)
- 对象的初始化顺序

关 系		Java中的表现与实现	
泛化（Generalization）		extends继承	
实现（Realization）		implements实现	
关联	组合（Composition）	某个类以成员变量的形式出现在另一个类中	整体与部分，二者不可分 例如学生和学生证
	聚合（Aggregation）		整体与部分，相对独立 例如学校和学生
			其他
依赖（Dependency）		某个类以局部变量或参数的形式出现在另一个类中	

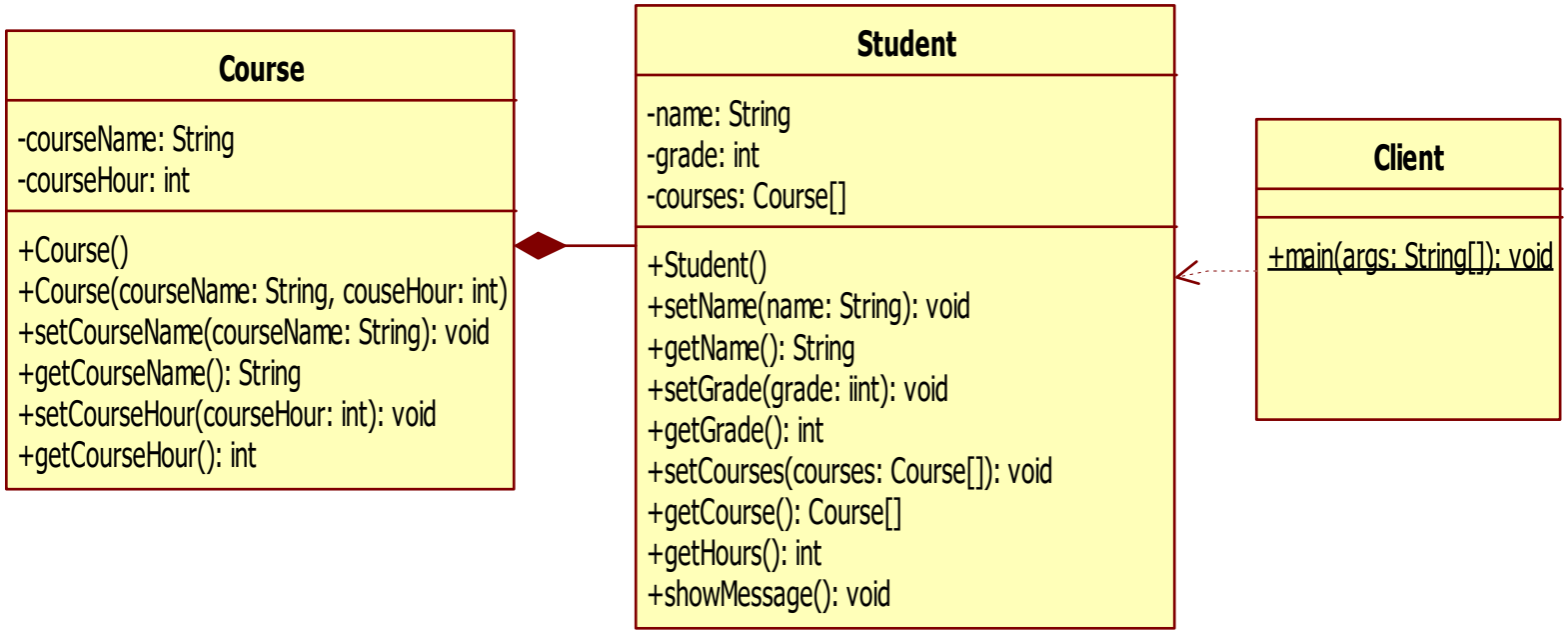
从泛化→实现→组合→聚合→关联→依赖，关系越来越弱。

5.1类与类的关系

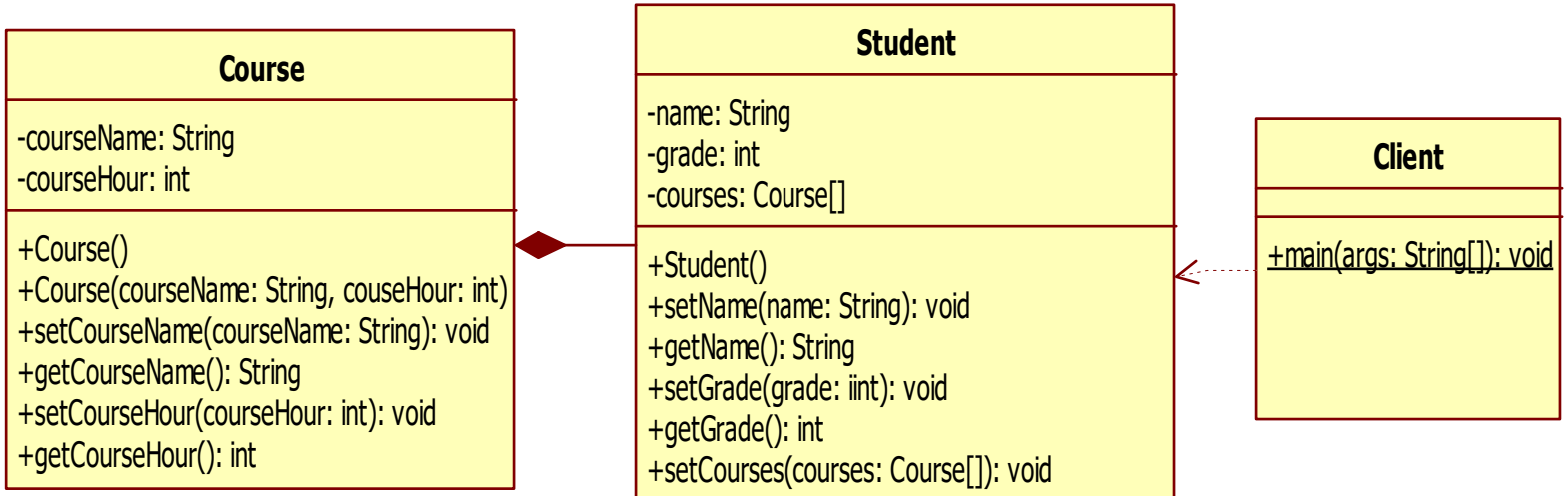
- 组合关系：“has a”，一个类的成员可以是其他类的对象引用
- 组合是OO设计中大量存在的类间关系。



5.1类与类的关系

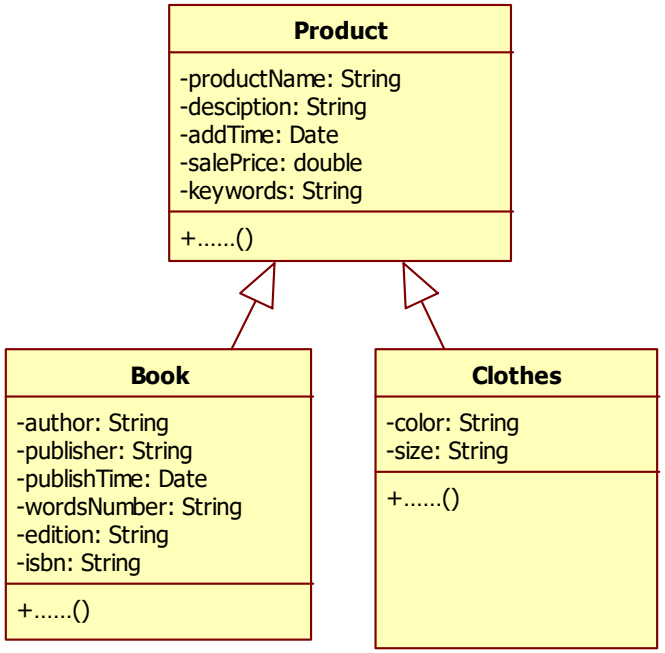


- UML图：Unified Modeling Language 统一建模语言
- 展示面向对象中的类之间的关系，每种关系的表达都有特定的符号。

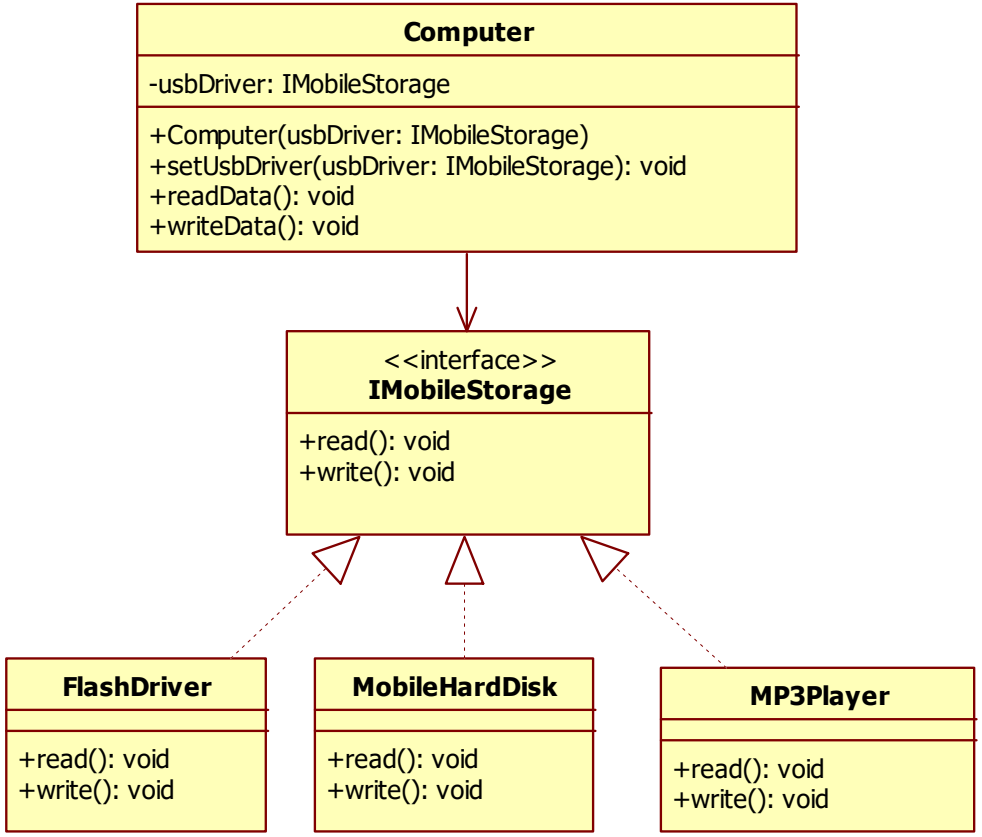


组合：整体与部分，二者不可分

依赖：某个类以局部变量或参数的形式出现在另一个类中



泛化：extends继承



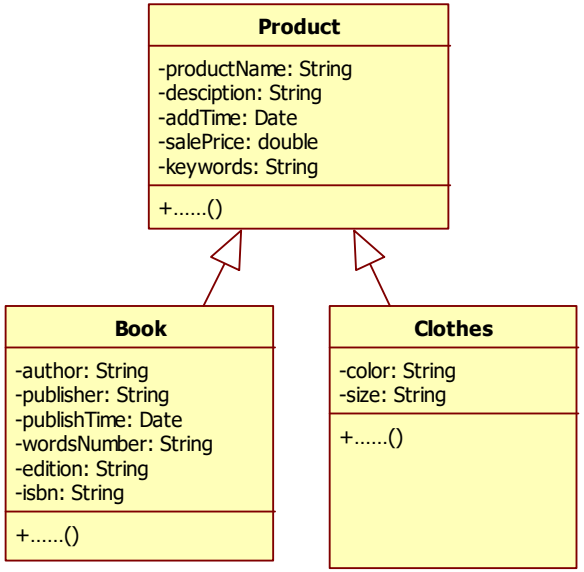
实现：implement

5.2.1 继承的概念

- 代码复用的手段
  - “is-a” 关系: 类之间是继承的关系
  - “has-a” 关系: 类之间是组合的关系

- 继承：从已有的类中派生出新的类。
  - 新的类能吸收已有类的数据成员和行为
  - 并能扩展新的能力

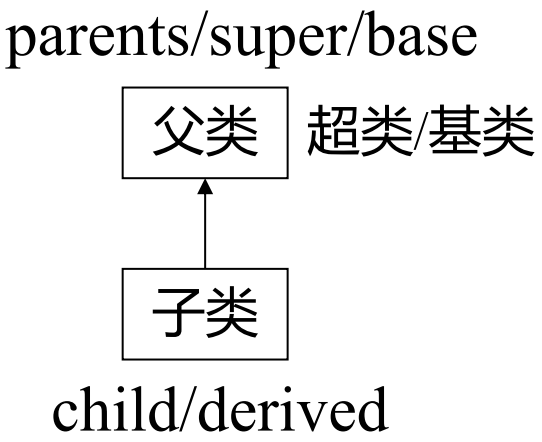
  - 举例：商品



5.1.1 继承的概念

- 父类是子类的一般化，子类是父类的特例化(具体化)。
- 父类也称为超类或基类。

父 类	子 类
学生	研究生、大学生、小学生
形状	三角形、圆、矩形
雇员	教师、医生、职员
交通工具	轿车、卡车、公交车
水果	苹果、梨、桃、桔



5.1.1 继承的概念

- 继承分类
  - 单继承：一个子类最多只能有一个父类。
  - 多继承：一个子类可有两个以上的父类。

- Java类只支持单继承，而接口支持多继承。Java多继承的功能则是通过接口方式和内部类来间接实现的。

- 继承的语法

```
[类修饰符]  class 子类名 extends 父类名{
        成员变量定义;
        成员方法定义;
}
```

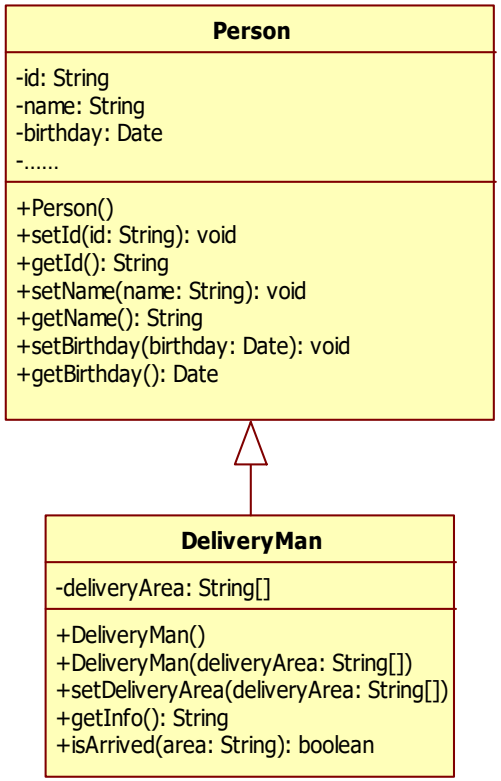
- 在子类的定义中，用关键字extends来明确指出它所继承的父类。

【例5-1】通过继承Person类派生DeliveryMan类。

快递公司的不同职工

职工Person
快递员 DeliveryMan

- 数据成员配送区域deliveryArea
- 获取配送区域方法getDeliveryArea()
- 获取快递员信息方法getInfo()
- 判断某区域是否在配送范围内方法isArrived()

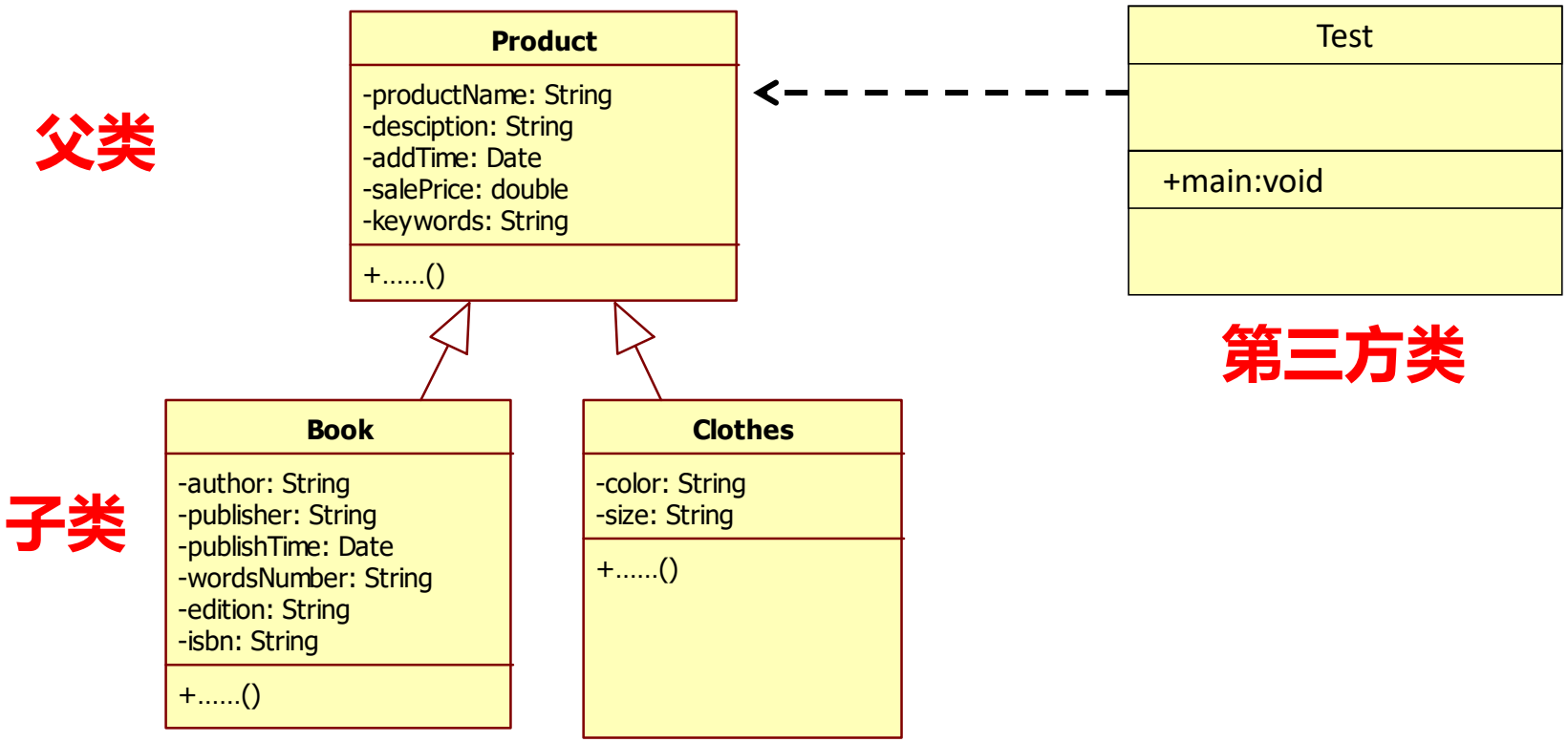


- 成员访问控制修饰符在继承中的性质
  - public、private、package、protected

对象.成员　　直接使用

类A成员的访问控制符	类A对类A成员的访问权限	第三方类对类A成员的访问权限		子类B对父类A成员的访问权限	
		与A同包	与A不同包	与A同包	与A不同包
public	√	√	√	√	√
protected	√	√	×	√	√
默认(package)	√	√	×	√	×
private	√	×	×	×	×

5.1.3 类成员的访问控制



- 成员访问控制修饰符的具体应用
  - 父类中数据成员的修饰符：尽可能应该将父类中的成员变量声明为private，并在父类中定义访问这些private成员变量的public型的方法。
  - 父类中方法的修饰符：关于父类中的private方法，如果某个方法是为类中其他的方法提供服务，只在类中使用，将其定义为private，对外界隐藏。

尽可能的private

Sample： IceCream.java

- 子类从父类继承成员时，父类的所有public、protected、package成员，在子类中都保持它们原有的访问修饰符。

- 注意**The default package：**

Sample： Cake.java Pie.java

【例5-2】分析同包下无继承关系的类之间成员的访问控制权限。

```
package chap5.example.access;
public class C {
        public int a=1;
        protected int b=2;
        int c=3;
        private int d=4;
        public int getD(){return d;}
}
```

```
package chap5.example.access;
public class AccessDemo {
        public static void  main(String[] args) {
                C coo = new C();
                System.out.println(coo.a);
                System.out.println(coo.b);
                System.out.println(coo.c);
                //System.out.println(coo.d);

                System.out.println(coo.getD());
        }
}
```

【例5-3】分析不同包下子类对父类成员的访问控制权限。

```
package chap5.example.access.sub;
public class C {
        public int a;
        protected int b;
        int c;
        private int d;
        public int getD(){return d;}
}
```

```
package chap5.example.access;
import chap5.example.access.sub.C;
public class AccessDemo extends
C{
        public void getInfo(){

        System.out.println(a);

        System.out.println(b);

        System.out.println(c);

        System.out.println(d);
        }
}
```

单例模式：一个类只能创建一个对象。
如何实现单例模式?

Sample： Lunch.java

访问权限修饰符还可以放在Class前

java文件中Class的访问权限修饰符只有public 和friendly，一个java文件只能有一个public 类，且该类的类名和文件名一致。

类型	无修饰（默认）	public
同一包中的类	是	是
不同包中的类	否	是

- 方法的重写：

在子类中如果创建了一个与父类中相同名称、相同参数列表的方法，只是方法体中的实现不同，以实现不同于父类的功能，这种方式被称为方法重写（override），又称为方法覆盖。

【例5-4】定义Animal类的子类Bird，并重写它的move()方法。

```
public class Animal {
        public void move(){
                System.out.println("我可以move...");
        }
}
public class Bird extends Animal{
        public void move(){
                System.out.println("我可以在天空飞翔.....");
        }
        public staticvoid main(String[] args){
                Bird bird = newBird();
                bird.move();   //输出"我可以在天空飞翔....."
        }
}
```

- 方法的重写遵循“两同两小一大”的规则
  - “两同”：方法名称相同、形参列表相同
  - “两小”
    - 子类方法返回值类型≤父类方法返回值类型
    - 子类方法抛出的异常≤父类方法抛出的异常
  - “一大”：子类方法的访问权限≥父类方法的访问权限

- 重写方法时不能改变方法的static或非static性质。
- 重写的方法可以使用 @Override 注解来标识。

5.2.2 Object类与重写toString()方法

北京理工大学 BEIJING INSTITUTE OF TECHNOLOGY

- Object类：Java中所有类的父类，定义和实现了Java系统下所有类的共同行为，所有的类都是由这个类继承、扩充而来的。
- 认识Object类中的方法
- https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/lang/Object.html
  - clone()
  - finalize()
  - toString()
  - equals()
  - hashCode()
  - notify()
  - notifyAll()
  - wait()
  - getClass()

5.2.2 Object类与重写toString()方法

北京理工大学 BEIJING INSTITUTE OF TECHNOLOGY

- toString()方法：对对象的文字描述，返回一个字符串。
- 如果在新创建的类中重写toString ()方法，返回一个与该类具体相关的字符串，则打印对象的操作将变得非常简单，直接调用System.*out*.println()方法即可。正因为如此，toString()方法通常都会被重写。

【例5-1】在DeliveryMan类中重写toString方法。

```
public class DeliveryMan extends Person {
        private String[] deliveryArea;
……
        public String toString(){
                String str = getId()+","+getName()+"\n配送范围:";
                int i;
                 for(i=0; i<deliveryArea.length-1; i++){//除最后一个外，
都有一个逗号
                        str+=deliveryArea[i]+",";
                }
                return str+deliveryArea[i];
        }
}
```

【例5-1】在DeliveryMan类中重写toString方法。

```
public class Test {
        public static void main(String[] args) {
                DeliveryMan dm = new DeliveryMan();
                dm.setId("007");
                dm.setName("Bang");

                dm.setDeliveryArea(new String[]{"南锣鼓巷","烟袋斜
街","雨儿胡同","帽儿胡同","黑芝麻胡同"});
                System.out.println("快递员信息:"+dm);
        }
}
```

5.2.3 调用父类被重写的方法

- 如果子类重写了父类的方法，那么在子类对象默认调用的是自己重写后的方法，调用父类被重写了的方法需要使用关键字super。
- super是Java提供的一个关键字，用于对象调用它从父类继承得到的数据成员或方法）。

- 与this一样，super也不能出现在static修饰的方法中。

【例5-4】利用super调用父类的同名方法。

```
public class Animal {
        public void move(){
                System.out.println("我可以move...");
        }
}
public class Bird extends Animal{
        public void move(){
                super.move();     //调用父类的move()方法
                System.out.println("我可以在天空飞翔.....");
        }
        public static void main(String[] args){
                Bird bird = new Bird();
                bird.move();   //输出"我可以move...我可以在天空飞
翔....."
        }
}
```

datamember hide：

子类中声明了与超类中相同的成员变量名。可以用super关键字访问从父类继承的成员变量super.datamember

Sample:HideDataMember　HideStaticDataMember(com.hn.cn）

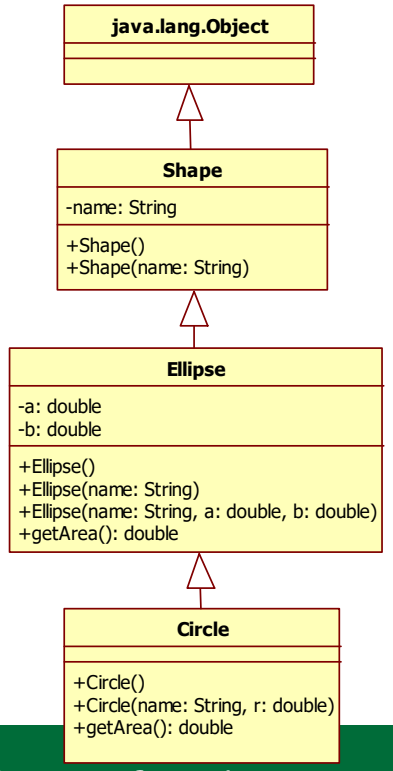
Method hide: 如果父类中的方法被重载多次，在子类中重写时不会隐藏父类重载的方法。

Sample : Hide. java

- **继承下的构造方法的调用次序**
  - 子类构造方法在执行自己的任务之前，将**显式**地(通过super引用)或**隐式**地(调用父类默认的无参数构造方法)调用其直接父类的构造方法。
  - 类似地，如果父类派生于另一个类，则要求父类的构造方法调用上一级类的构造方法，依此类推。调用请求中，最先调用的一定是Object类的构造方法。
  - 创建对象的过程：先父后子。

When you create an object of the derived class, it contains a *subobject* of the base class.

【例5-8】已知圆Circle→椭圆Ellipse→形状Shape的继承关系，查看Circle对象构建的过程。




```
public class Shape {
        private String name;
        public Shape() {                    super();
                System.out.println("Shape()...");
        }
        ......
}
public class Ellipse extends Shape{
        private double a; //短轴
        private double b;  //长轴
        public Ellipse() {                  super();
                System.out.println("Ellipse()...");
        }
        ......
}
public class Circle extends Ellipse{        super();
        public Circle() {
                System.out.println("Cicle()...");
        }
}
```

如果子类构造方法体中没有显式调用超类构造方法，则系统在执行子类的构造器时自动调用父类的无参构造器（自定义或默认的）

为了避免错误，建议父类中至少定义一个无参的构造方法。

```
public class Test {

    public static void main(String[] args) {
        new Circle("大圆",100);
    }
}
```

Shape(String)...
Ellipse(String)...
Ellipse(String,double,double)...
Circle(String,double)...

子类构造方法隐式调用父类无参构造方法

Example：DogTest

```
class Pet{
    private String name;
    public Pet(){
        System.out.print(1);
    }
    public Pet(String name){
        System.out.print(2);
    }
}
class Dog extends Pet{
    public Dog(String name){
        System.out.print(3);
    }
}
class Test{
    public static void main(String[] args) {
        new Dog("秋田");
    }
}
```

子类构造方法隐式调用父类无参构造方法


```
class Pet{
    private String name;
    public Pet(){
        System.out.print(1);
    }
    public Pet(String name){
        System.out.print(2);
    }
}
class Dog extends Pet{
    public Dog(String name){
        super();
        System.out.print(3);
    }
}
class Test{
    public static void main(String[] args) {
        new Dog("秋田");
    }
}
```

5.3.2 super与this调用构造方法

- 子类不会继承父类的构造方法，但是子类可以调用父类的构造方法，如同一个类的构造方法可以用this()调用自己的重载构造方法一样。子类调用父类构造方法使用super()完成。

5.3.2 super与this调用构造方法

```
public class Shape {
        private String name;
        public Shape() {
                System.out.println("Shape()...");
        }
        public Shape(String name) {
                this.name = name;
                System.out.println("Shape(String)...");
        }
}
```

5.3.2 super与this调用构造方法


```
public class Ellipse extends Shape{
        private double a; //短轴
        private double b;  //长轴
        public Ellipse() {
                System.out.println("Ellipse()...");
        }
        public Ellipse(String name) {
                super(name);          //调用Shape的带参构造方法
                System.out.println("Ellipse(String)...");
        }
        public Ellipse(String name, double a, double b) {
                this(name);   //调用本类的重载构造方法
                this.a = a;
                this.b = b;
                System.out.println("Ellipse(String,double,double)...");
        }
}
```

5.3.2 super与this调用构造方法

```
public class Circle extends Ellipse{
        public Circle() {
                System.out.println("Cicle()...");
        }
        public Circle(String name, double r) {
                super(name,r,r);   //调用Ellipse的构造方法
                System.out.println("Circle(String,double)...");
        }
}
```

- **5.4.1 final修饰符**
  - final修饰类时，类不能被继承
  - final修饰方法时，方法不能被重写
  - final修饰变量时，变量只能被赋值一次

1. final类

- final修饰的类不能有子类。
- 像java.lang.String、java.lang.Math等就是final类，不能被继承，它们的方法禁止被重写。
- 如果试图对该类进行继承，则会出现编译错误。

2. final方法

- final修饰的方法不能被重写，
- 如java.lang.Object类中的getClass()方法。Object类是一定会被继承的（它是所有类直接或间接的父类），但是Java不希望子类重写这个方法，所以使用final把它保护起来。Object中的final方法还有notify()、notifyAll()和wait()。

3．final变量

- 无论final修饰哪种变量，无论这种变量在何处被赋初值，final变量的赋值永远只能有一次。
  - final成员变量
  - final局部变量
  - final方法参数

【finalDemo】

static final 变量Sample：FinalData.java

5.4.2 Java修饰符之间的关系

北京理工大学 BEIJING INSTITUTE OF TECHNOLOGY

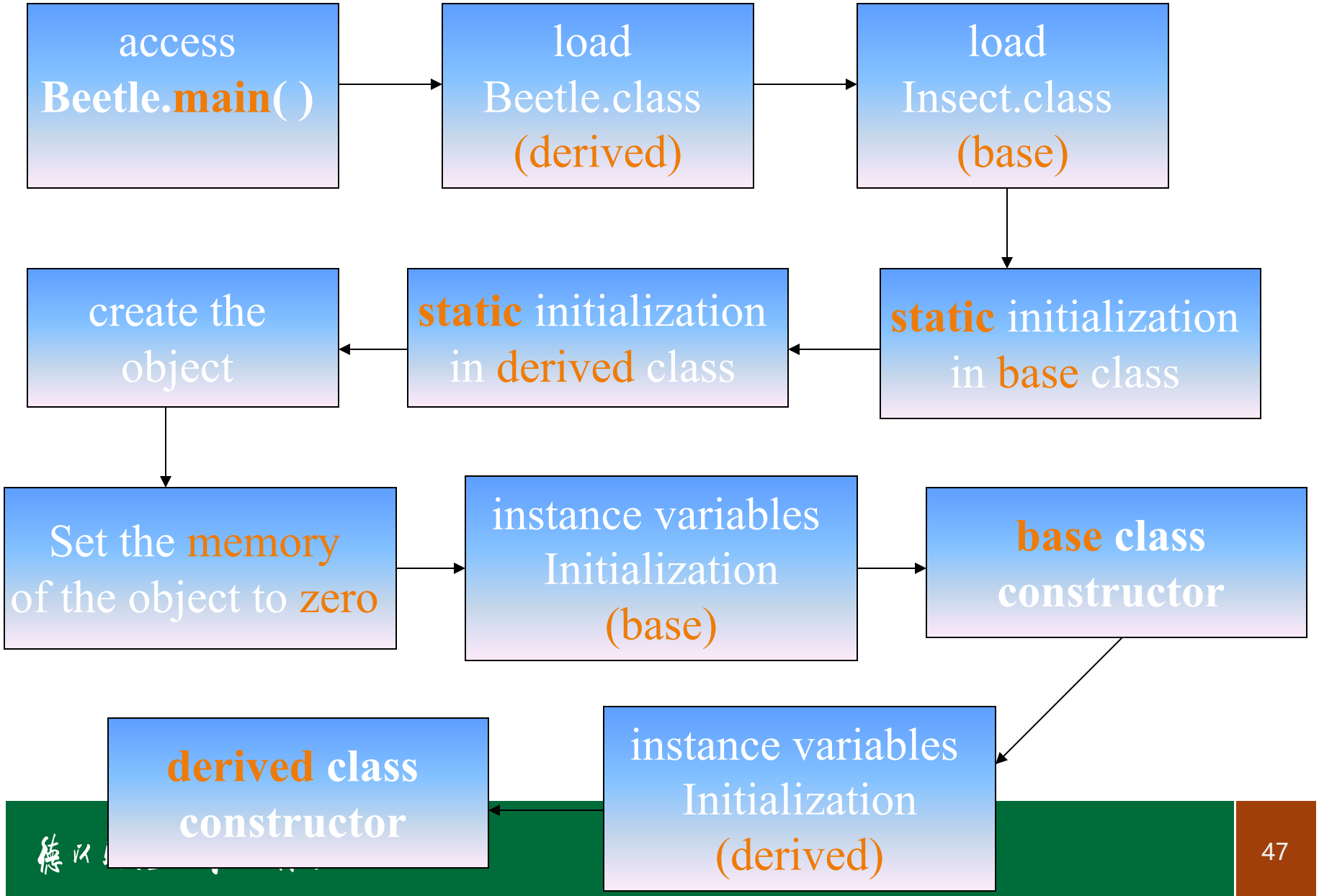
	public	protected	默认	private	static	final
类	√		√			√
数据成员	√	√	√	√	√	√
方法成员	√	√	√	√	√	√
局部变量						√

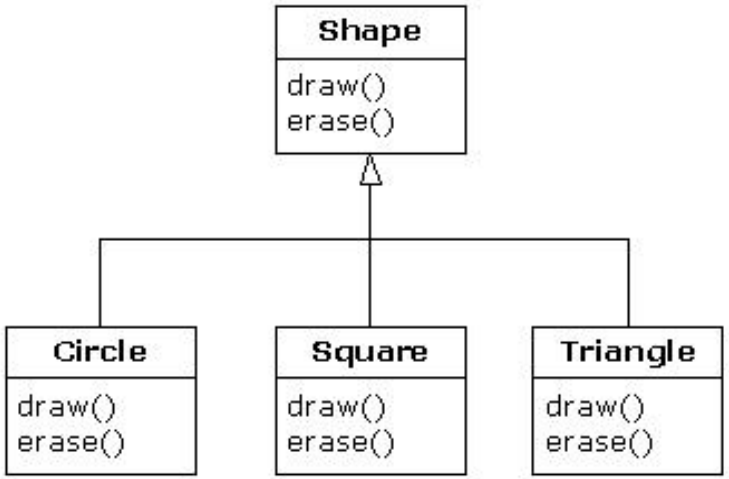
- 继承重用的是形式。
- 组合重用的是功能

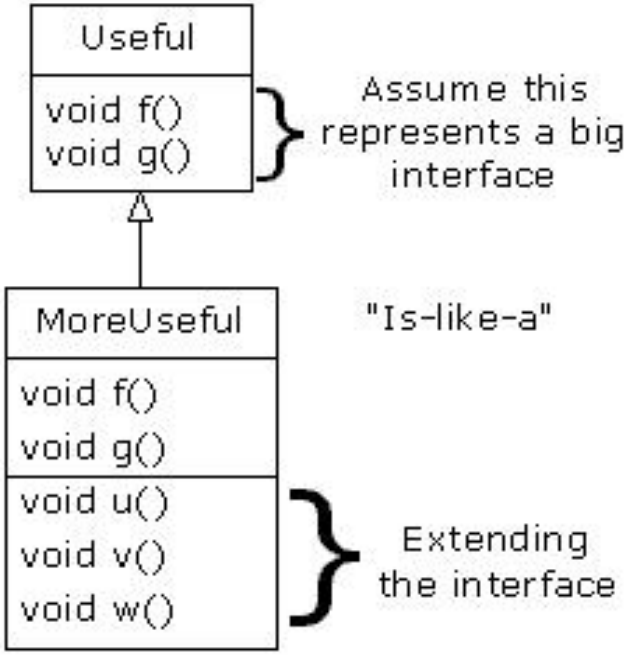
is-a ---- inheritance

has-a ---- composition

5.6类的初始化Sample：Beetle. java



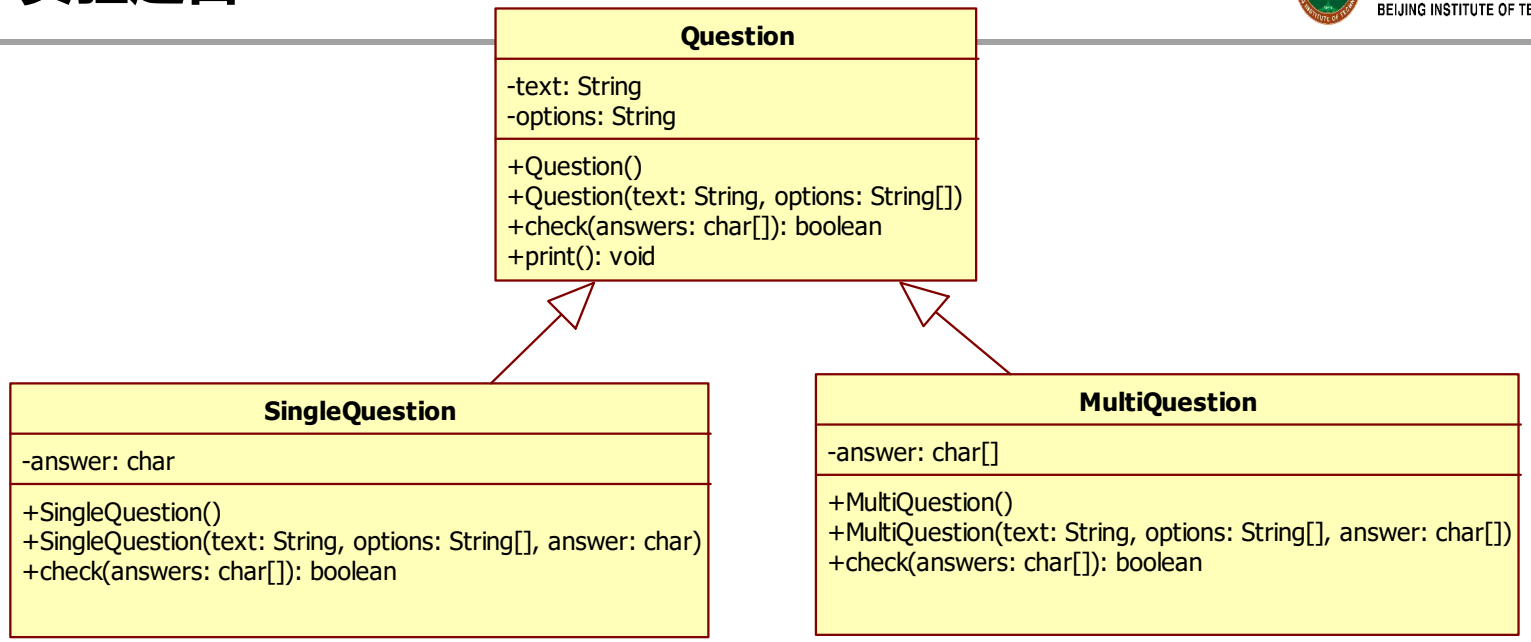

Shape
draw()
erase()
Circle
draw()
erase()
Square
draw()
erase()
Triangle
draw()
erase()


Useful
void f()
void g()
Assume this represents a big interface
MoreUseful
void f()
void g()
void u()
void v()
void w()
"Is-like-a"
Extending the interface

is-a

is-like-a

实验题目2



三国演义中的三绝是谁?
A.曹操　　B.刘备　C.关羽　D.诸葛亮
请选择:cad
恭喜, 答对了!

最早向刘备推荐诸葛亮的是谁?
A.徐庶　　B.司马徽　　C.鲁肃　D.关羽
请选择:A
还得努力呀!